



Nowcasting Modelling with IRENE

Module 2 · Forecasting Precipitation with AI and Radar Data

IT4LIA AI Factory · 23 September 2026 · 09:00–13:00

Today's agenda

Time	Segment
09:00 – 09:30	Welcome & review of the setup
09:30 – 09:45	The IT-DPC-SRI dataset
09:45 – 10:00	Nowcasting and the IRENE model
10:00 – 10:45	Hands-on: how to use IRENE
10:45 – 11:00	Break
11:00 – 11:15	PyTorch foundations & training an AI model
11:15 – 11:45	Notebook: PyTorch foundations & training
11:45 – 12:00	Break
12:00 – 12:45	IRENE training pipeline & optimization
12:45 – 13:00	Wrap-up & Q&A

Materials

Repository folder
`module2-irene-nowcasting/`

`notebooks/ · convgru_ensemble/`

Prerequisites

Python, PyTorch basics
ML/DL fundamentals
Module 1 (or its .env set up)

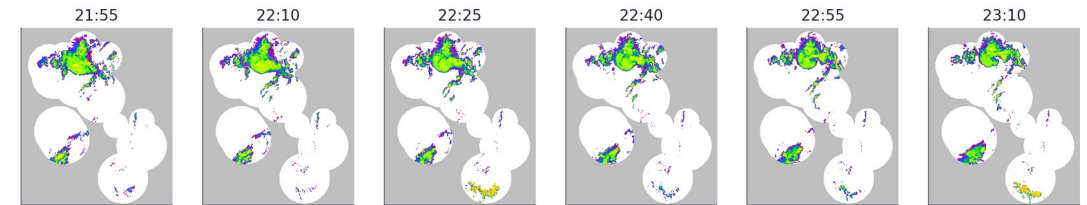
PART 1

The IT-DPC-SRI dataset

The archive behind everything today: what is in it, how to connect, how to pull out the frames you need.

Recap from Module 1: the data

Property	Value
Product	DPC Surface Rainfall Intensity
Variable	RR — rain rate, mm/h
Grid	1400 × 1200, ~1 km
Cadence	5 min (since Jul 2020; 10 min 2014-2020, 15 min before)
Coverage	~half the grid is NaN
Storage	Zarr on ArcoDataHub, chunks (1, 1400, 1200)



the archive, 28 Aug 2025 — consecutive frames, 15 min apart

Task definition

Input: past rain-rate frames (T, H, W)

Output: ensemble forecast (E, T', H, W)

IRENE: 6 past frames (30 min) → 12 future frames (60 min)

See it in practice → [Notebook 01 • 2. Get the data](#)

Connecting to the archive

```
open the store

url = f"https://{user}:{key}@api.arcodatahub.com/S3/italian-radar-dpc-sri.zarr"

ds = xr.open_zarr(url)  # metadata only
```

Before you start

Create an account on arcodatahub.com, verify the e-mail, then subscribe to the `italian-radar-dpc-sri.zarr` dataset.

Always check `last_valid`

The time axis is pre-allocated to 2049, so most of it is empty. Everything after `ds.attrs["last_valid"]` is NaN.

See it in practice → [Notebook 01 · 2. Get the data](#)

Getting the data you need

```
time_start = "2025-08-28T22:25"
time_end   = "2025-08-28T23:25"

# by time
data = (ds["RR"].sel(time=slice(time_start, time_end)) # by label
        .isel(time=slice(100, 200))                  # by index
        .load())                                     # download
)

# a region (x and y are metres)
patch = data.sel(
    x=slice(-180_000, 204_000),
    y=slice(600_000, 216_000)
)
```

- Select by label (sel) or position (isel)
- x, y are metres in the archive's projection
- NaN means "no data"
- One chunk is one step, whole space

See it in practice → [Notebook 01 · 2. Get the data](#)

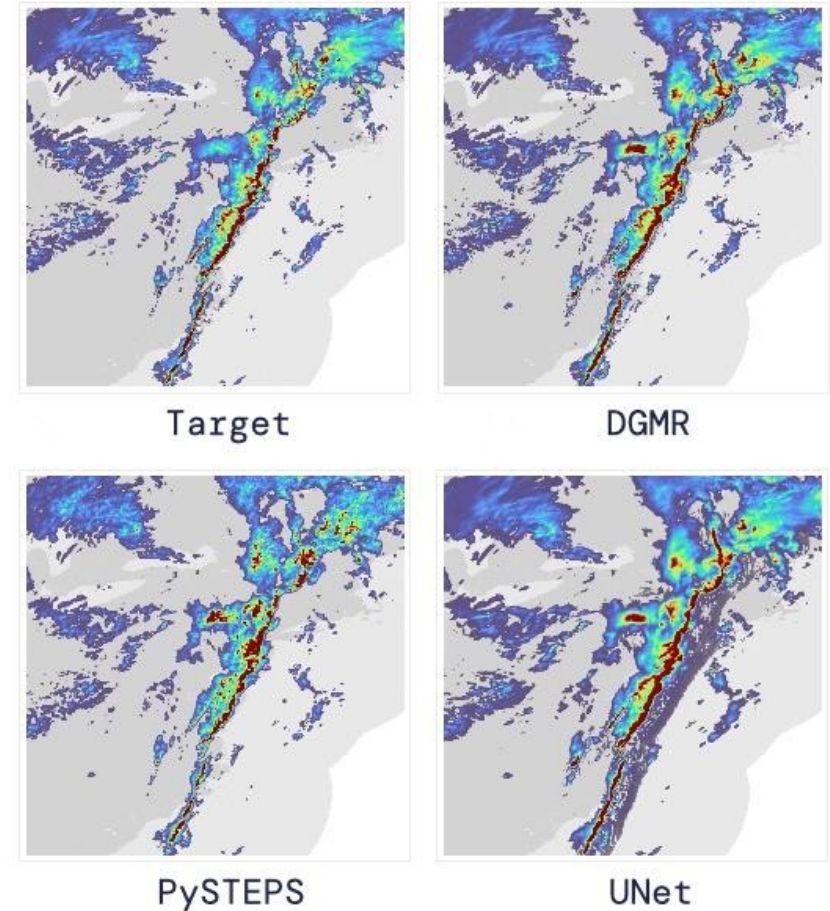
PART 2

Nowcasting and the IRENE model

What a nowcast is and what is IRENE.

Nowcasting

- Forecast weather at **high resolution** for the next **minutes to hours**
- Radar estimates what is falling now. A nowcast says where it will be in 5 to 120 minutes
- **Uncertainty grows fast** → we want **ensembles**, not one deterministic field



Target · DGMR · PySTEPS · UNet — image from deepmind.google

Three ways to predict the next hour

Physics • NWP

- Solve the equations of the atmosphere
- Needs assimilation and spin-up: hours
- Convection at km scale is expensive
- Ready **too late for the next 60 minutes**

Extrapolation • STEPS

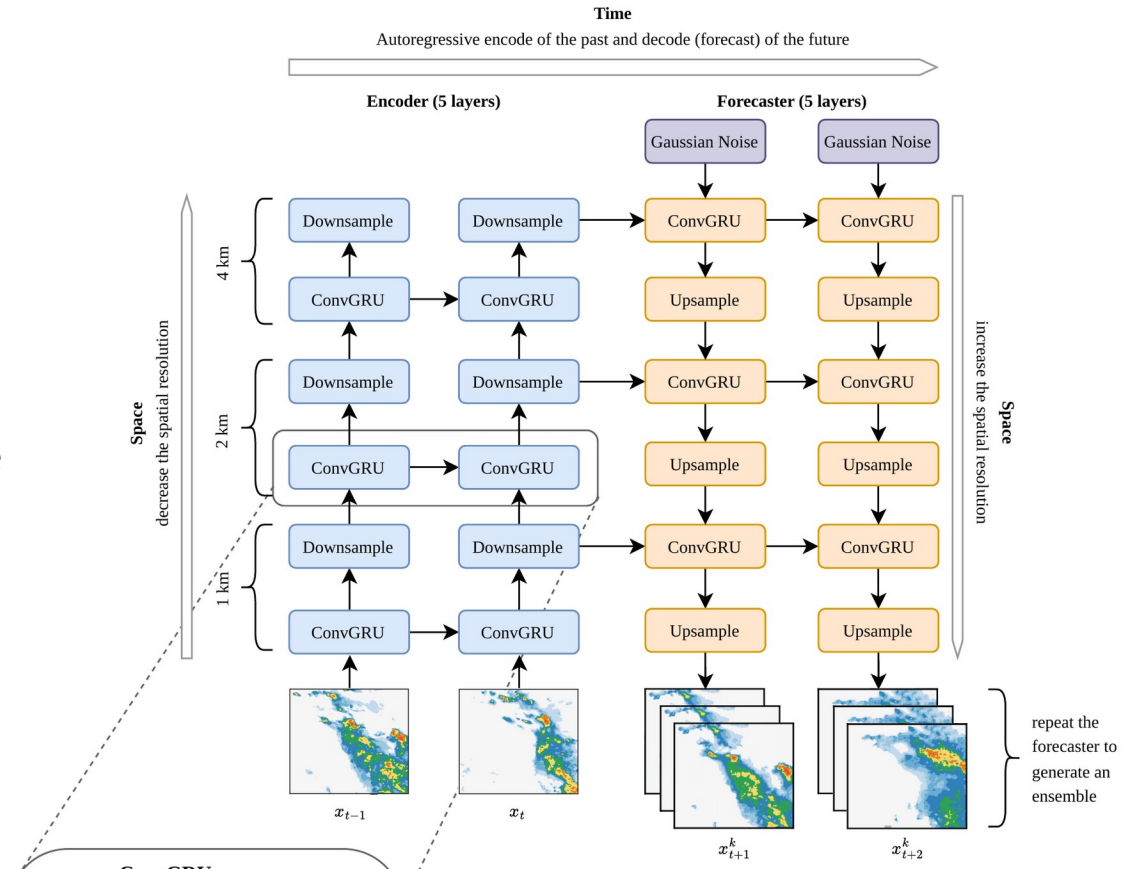
- Estimate the motion of the last frames
- **Advect the field**, perturb it with noise
- Seconds to run, operational standard
- Rain neither grows nor decays

Learned • IRENE

- Learn the evolution from years of radar
- Growth and decay come from the data
- Ensemble by construction, seconds to run
- Only as good as the archive it saw

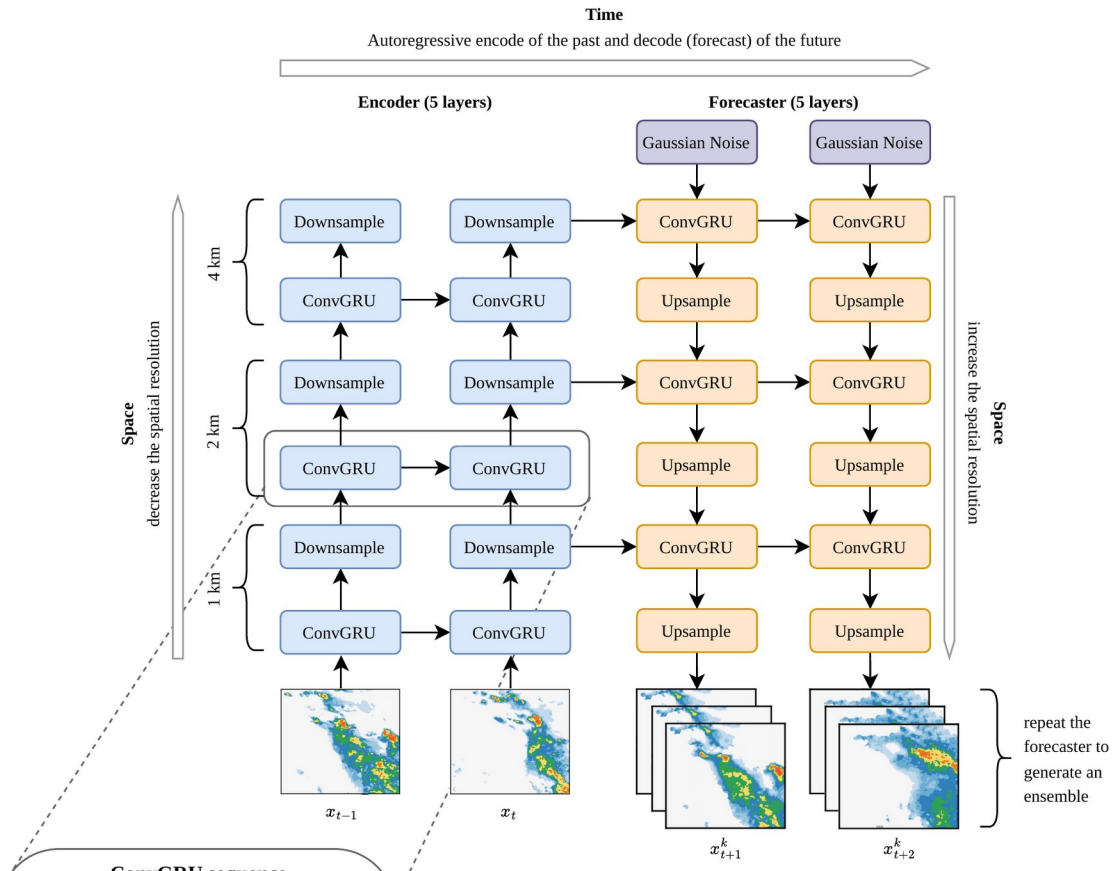
IRENE at a glance

- Italian Radar Ensemble Nowcasting Experiment
- ConvGRU encoder-decoder with PixelUnshuffle / PixelShuffle
- Probabilistic: an ensemble of plausible futures
- Trained with the almost-fair CRPS on the DPC SRI archive
- **64.2 M** parameters
- Developed at FBK within IT4LIA
- Available on Hugging Face: [it4lia/irene](https://huggingface.co/it4lia/irene)



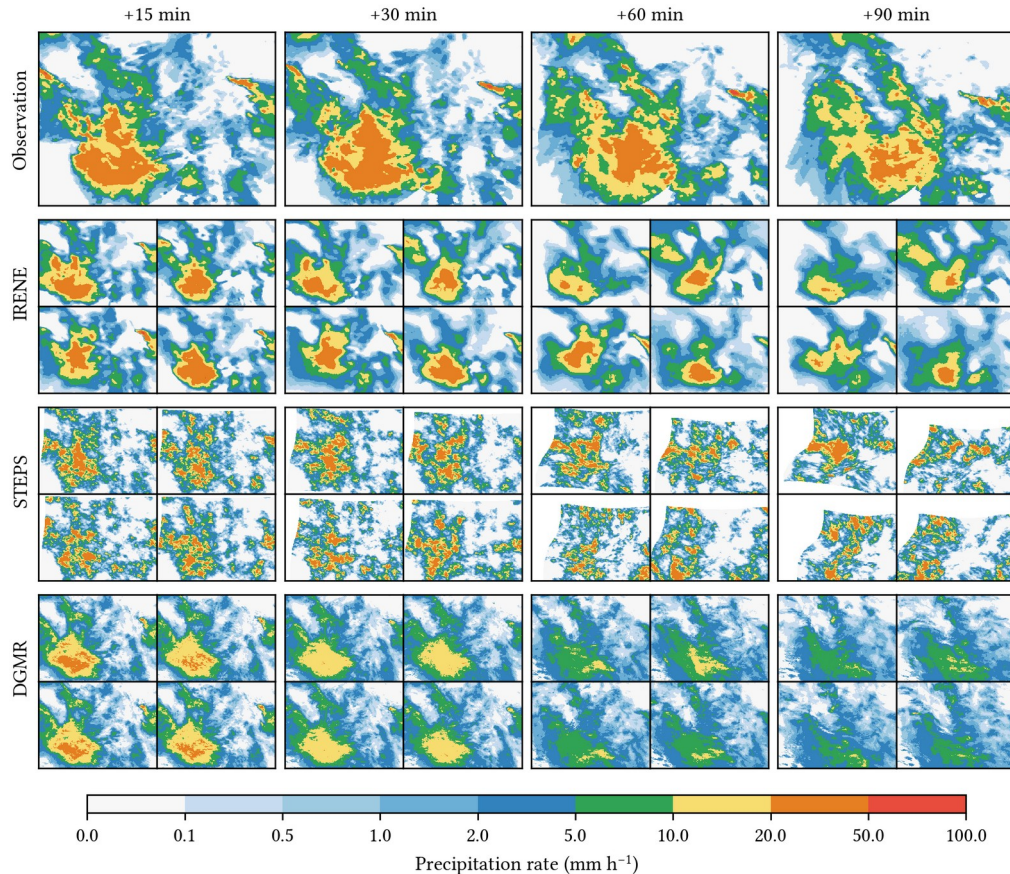
See it in practice → [Notebook 01 · 1. Get the model](#)

Inside IRENE



- **Encoder:** the past frames climb 5 ConvGRU levels, each halving the resolution — 1 km up to 32 km
- **Forecaster:** mirrors it and unrolls the future one frame at a time, upsampling back to 1 km
- **Gaussian noise** feeds the forecaster — run it again with fresh noise and you get another member
- **ConvGRU cell:** a GRU whose gates are convolutions, so the hidden state stays an image (appendix)
- **64.2 M parameters** · 6 frames in (30 min) → 12 out (60 min)

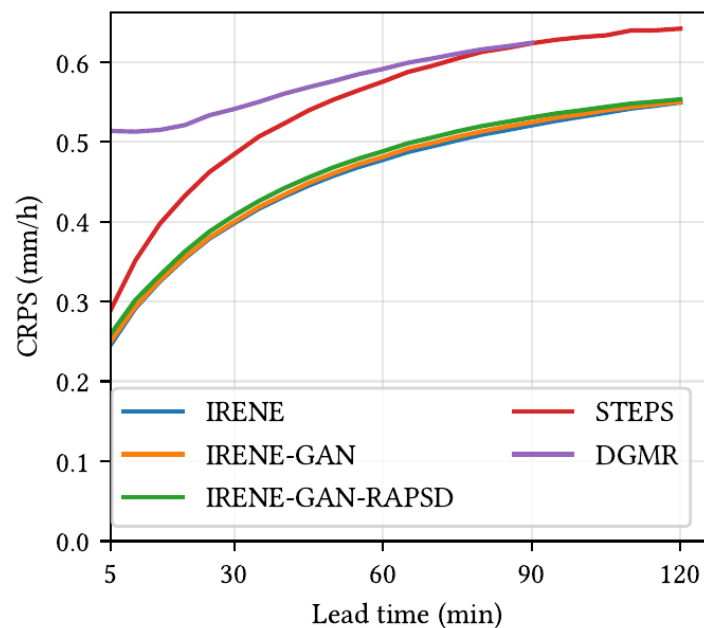
How does IRENE compare?



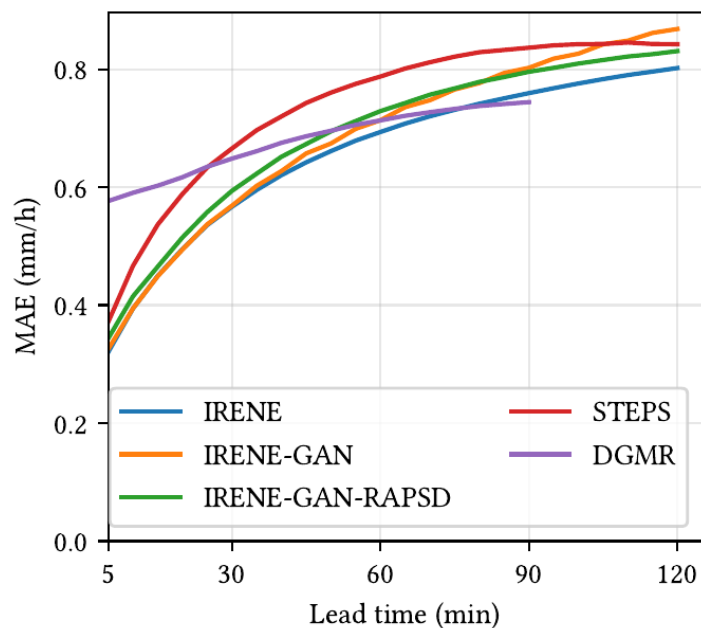
IRENE paper · 256 km patch, Ligurian Sea, 2 Nov 2025

- One event, +15 to +90 min: the observation on top, then four ensemble members per model
- **IRENE** keeps the large-scale field but smooths the detail — the price of a CRPS-type loss
- **STEPS** advects what it sees: structure survives, but nothing grows or decays; the blank corners are pixels it has no source for
- **DGMR** (used outside its training domain, without fine-tuning) damps the field away after ~40 min
- The paper also trains two adversarial variants that keep small-scale texture — appendix

Scores: probabilistic skill



CRPS — lower is better



MAE of the ensemble mean

- **CRPS:** all three IRENE variants beat STEPS and DGMR at every lead time out to 120 min
- Rank histograms are much closer to uniform — the ensembles are better calibrated
- **MAE rewards smooth fields.** DGMR wins at +90 min precisely because it has damped the rain away.
- **STEPS** is strong in the first half hour, then degrades and underpredicts the heaviest rates

IRENE paper (Camilletti, Franch et al., in preparation) — evaluated on the held-out test period

PART 3

Hands-on: how to use IRENE

Checkpoint, live data, predict() — then the knobs, the region, and what an ensemble buys you.

PART 4

PyTorch foundations & training an AI model

Tensors, autograd, models, data loading and the training loop — the vocabulary behind IRENE.

The machine-learning pipeline



training → validation → tuning is a loop, not a line — it runs many times before testing

Where the time actually goes

Collection and preprocessing dominate real projects. Module 1 is entirely the first two boxes.

Where this part sits

Module 2 is the middle of the chain: how a model is defined, trained and validated in PyTorch. The released checkpoint is the deployment box.

Why PyTorch

Looks like Numpy

Zero learning curve for Python users

Autograd built in

You write the forward pass; the backward pass is derived for you.

One array type, CPU or GPU

Tensors move across devices with a `.to()` call, and share memory with numpy.

The ecosystem default

Most published models, weights and tutorials are PyTorch — IRENE included.

Tensors: arrays that know their device

```
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])

x.shape      # torch.Size([2, 2])
x.dtype      # torch.float32
x.device      # cpu

x = x.to("cuda")      # same data, on the GPU
t = torch.from_numpy(arr) # zero-copy on the CPU
```

- Shapes, slicing and broadcasting behave like numpy
- Two additions: a tensor can live on a GPU, and it can track gradients
- Every tensor in one operation must share a device
- numpy defaults to float64, PyTorch to float32 — convert explicitly

See it in practice → [Notebook 02 · 1. Tensors](#)

Autograd: gradients come for free

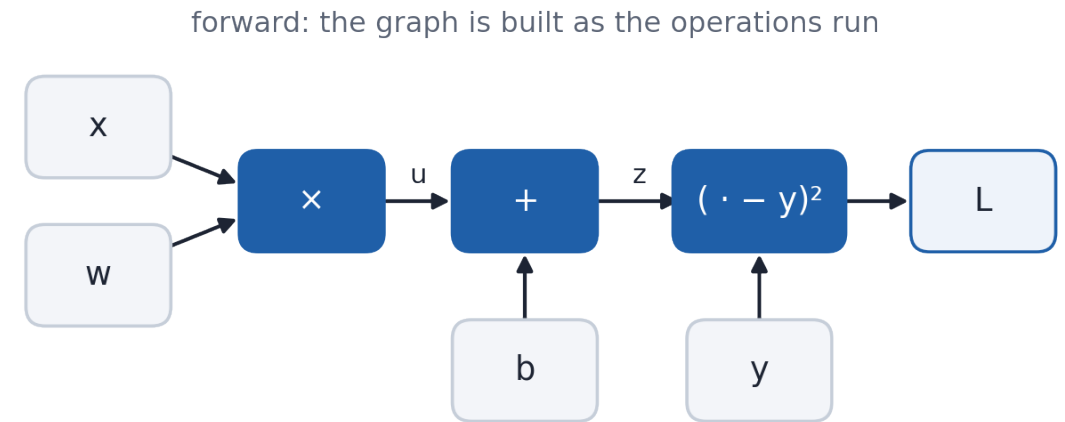
```
w = torch.tensor(3.0, requires_grad=True)

z = w * x + b          # every op is recorded
loss = (z - y).pow(2)   # a scalar

loss.backward()         # chain rule, backwards
                        # dL/dw = 2(z - y) * x

with torch.no_grad():   # inference: no graph
    pred = model(x)
```

- The graph is recorded as the forward pass runs, then freed by `backward()`
- Gradients accumulate — hence `optimizer.zero_grad()` every step
- `no_grad()` skips the bookkeeping: faster, and far less memory



backward() ←-----

$$\partial u / \partial w = x$$

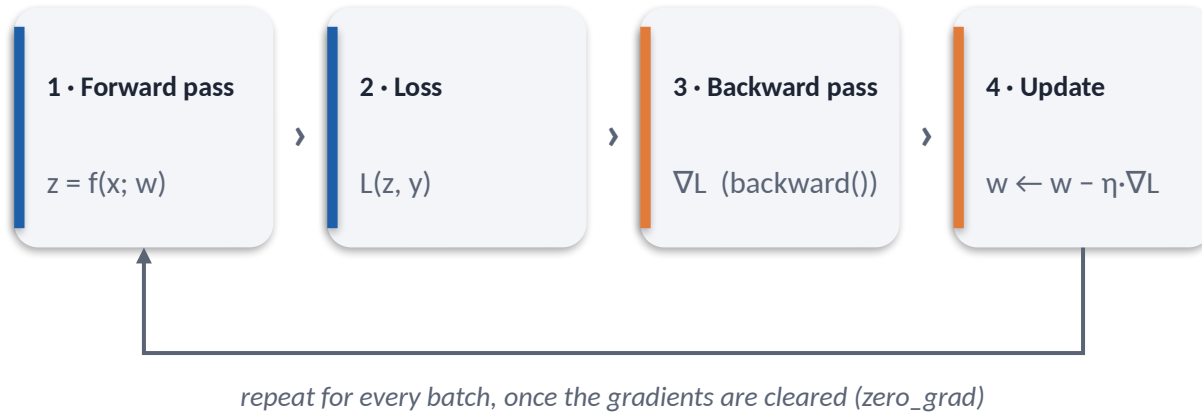
$$\partial z / \partial u = 1$$

$$\partial L / \partial z = 2(z - y)$$

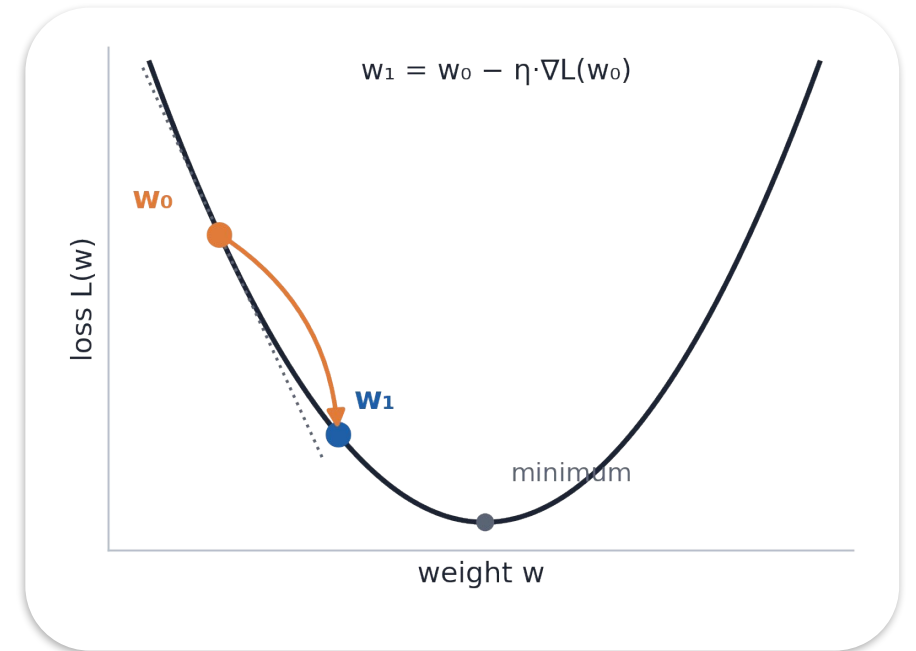
backward() multiplies the partial derivative on every edge

See it in practice → [Notebook 02 · 2. Autograd and weights update](#)

Gradient descent: updating the weights



One update step nudges every weight a little against its own gradient — a small step η (the learning rate) at a time, so the loss keeps decreasing without overshooting.



Try it

02_pytorch_dl_intro.ipynb · 2. Autograd and weights update — the forward / loss / backward / update steps above are exactly those cells, run one gradient step at a time.

nn.Module: a model is a class

```
class SimpleDenseNet(nn.Module):  
    def __init__(self, ...):  
        self.model = nn.Sequential(...) # layers  
  
    def forward(self, x): # (B, 1, 28, 28)  
        return self.model(x.flatten(1)) # (B, 10)
```

- `__init__` creates the layers, `forward` defines the computation
- Anything assigned to `self` is registered, so `.parameters()` finds it
- Call the module, never `forward()` directly
- No softmax at the end: `CrossEntropyLoss` expects raw scores
- IRENE's `EncoderDecoder` is this shape, with `ConvGRU` cells inside

See it in practice → [Notebook 02 · 3. The model: nn.Module](#)

Dataset and DataLoader

```
class CustomImageDataset(Dataset):  
    def __len__(self): ... # how many  
    def __getitem__(self, i): ... # sample i  
  
dl = DataLoader(trainset, batch_size=128,  
                shuffle=True, num_workers=2)
```

- Dataset answers two questions: how many samples, and give me sample i
- DataLoader adds batching, shuffling and worker processes
- random_split carves out train / val / test
- In IRENE one item is a (T, H, W) radar datacube cut from the Zarr archive
- Choosing which datacubes to sample is its own problem — see Part 4

See it in practice → [Notebook 02 · 4. Datasets and DataLoaders](#)

The training loop

```
for epoch in range(epochs):  
    model.train()  
    for X, y in train_dl:  
        X, y = X.to(dev), y.to(dev)  
  
        pred = model(X)           # 1. forward  
        loss = criterion(pred, y) # 2. loss  
  
        loss.backward()          # 3. backward  
        optimizer.step()         # 4. update  
        optimizer.zero_grad()    # 5. clear
```

- Five steps, unchanged in every supervised project
- train() / eval() switch dropout and batch-norm behaviour
- Wrap validation in torch.no_grad()
- CrossEntropyLoss for classification; IRENE uses CRPS, which scores an ensemble

See it in practice → [Notebook 02 · 2. Autograd and weights update](#)

From PyTorch to Lightning

What you write	What the Trainer takes
training_step	the epoch and batch loops
validation_step	backward / step / zero_grad
configure_optimizers	moving tensors to the device
the model itself	train() and eval() toggling
	checkpoints, logging, multi-GPU
	mixed precision, early stopping

the same training, with the loop deleted

```
class MNISTLitModule(pl.LightningModule):  
    def training_step(self, batch, idx): ...  
    def configure_optimizers(self): ...  
  
pl.Trainer(max_epochs=2).fit(model, datamodule=dm)
```

See it in practice → [Notebook 02 · 6. Lightning Modules](#)

PART 5

IRENE training pipeline & optimization

From a multi-terabyte archive to mini-batches: data selection, Lightning modules, loss and optimizer.

From archive to training samples

Zarr archive

(time, y, x)

NaN filter

filter_nan.py: $\leq$
n_nan missing, no
gaps

valid CSV

t, x, y candidates

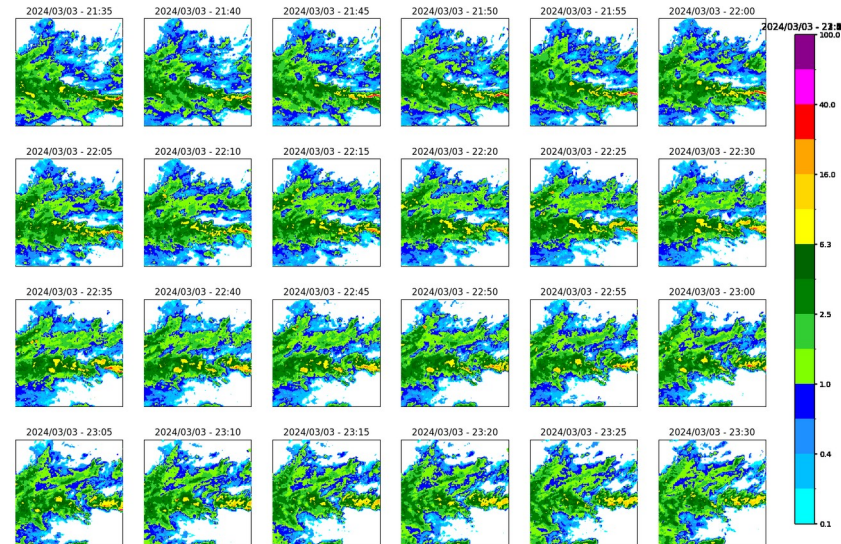
sampler

sample_valid_datacu
bes.py: keep with
prob. q

sampled CSV

importance_sampler/
output/

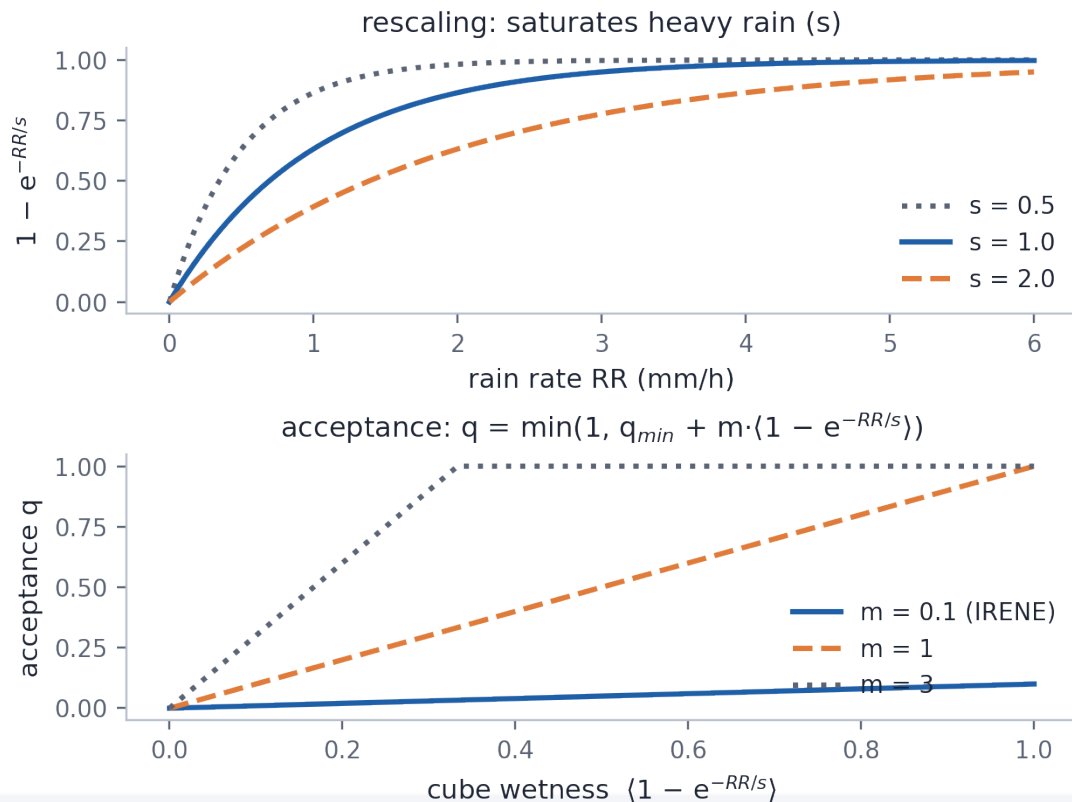
	x	y	t
0	0	0	432
1	0	16	90
2	0	16	378
3	0	32	180
4	0	32	327
...	...	...	...
227145	224	48	349458
227146	224	64	349008
227147	224	80	349062
227148	224	112	349164
227149	224	128	349077



- **Datacube:** $24 \times 256 \times 256$
- **Stride:** $3 \times 16 \times 16$
- **n_nan:** 10 000 pixels
- Each row = top-left corner of one cube

See it in practice → [Notebook 03 · 2. NaN cleaning](#)

Importance sampling: favouring rain



Parameter	IRENE value and effect
<code>q_min</code>	1e-4 — floor probability, keeps some dry cubes
<code>m</code>	0.1 — weight on wetness, the main knob
<code>s</code>	1 — saturation scale of $1 - \exp(-RR/s)$
<code>n_rand</code>	1 — draws per cube, allows oversampling

Try it

03_importance_sampling_demo.ipynb runs both scripts on one day of the live archive and compares $m = 0, 0.1, 1$.

Pre- and post-processing

mm/h

sampled rain
rate from the
archive

dBZ

$Z = 200 \cdot R^{1.6}$,
clipped to 0–60

[-1, 1]

dBZ / 30 - 1

model

ensemble
forecast

mm/h

inverse
transform,
crop

```
all of it, behind predict()
model = RadarLightningModel.from_pretrained("it4lia/irene")

past = radar[:6].values          # (T, H, W), mm/h
pred = model.predict(past, forecast_steps=12, ensemble_size=10)
pred.shape                       # (10, 12, H, W), mm/h
```

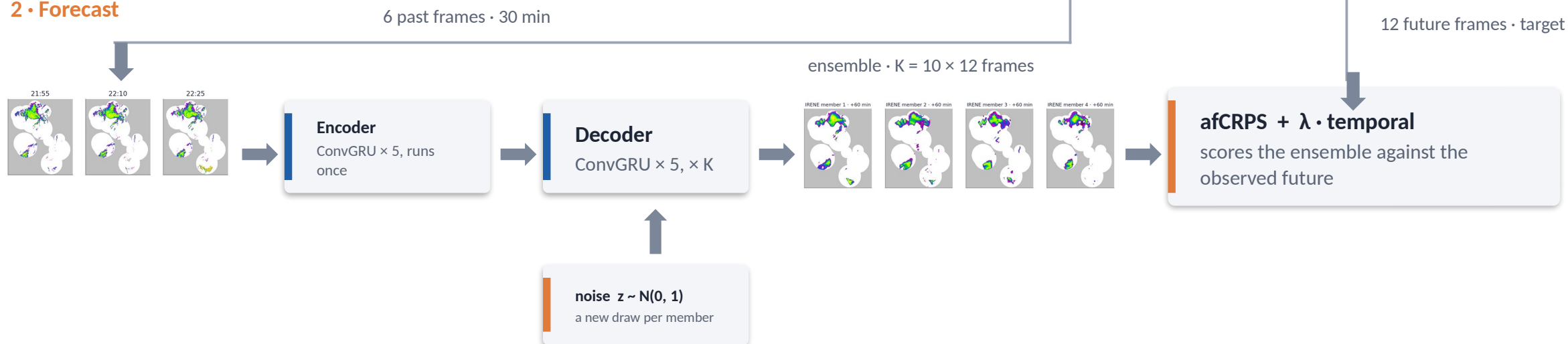
See it in practice → [Notebook 01 · 3. From the archive to predict\(\)](#)

How IRENE is trained, end to end

1 • Data selection



2 • Forecast



Loss: almost-fair CRPS

$$\widehat{\text{CRPS}}_K = \frac{1}{K} \sum_{k=1}^K |y^{(k)} - y| - \frac{1}{2K^2} \sum_{k=1}^K \sum_{j=1}^K |y^{(k)} - y^{(j)}|$$

$$\mathcal{L}_{\text{afCRPS}} = \frac{1}{2K(K-1)} \sum_{k=1}^K \sum_{j \neq k} \left(|y^{(k)} - y| + |y^{(j)} - y| - (1 - \varepsilon) |y^{(k)} - y^{(j)}| \right), \quad \varepsilon = \frac{1 - \alpha}{K}$$

accuracy: $1/K \sum |x_k - y|$

Every member should be close to the observation y .

spread: $-1/2K^2 \sum |x_k - x_j|$

Subtracted, so an ensemble that collapses to one value is penalised.
The almost-fair variant rescales it by $(1 - \varepsilon)$, $\varepsilon = (1 - \alpha)/K$, with $\alpha = 0.95$: it removes the finite-K bias.

+ λ • temporal

Mean $|x(\tau+1) - x(\tau)|$ over lead times and members, weighted by $\lambda = 0.01$: no frame-to-frame pulsing.

- **IRENE trains with `afcrps`** ($K = 10$ members, masked to valid pixels); `mse` • `mae` • `crps` also available (`loss_class`)
- **One member $\rightarrow$ CRPS = MAE.** Deterministic losses on uncertain targets give blurry forecasts.

See it in practice $\rightarrow$ [Notebook 04 • 2. Model](#)

Dataset: one sample = one datacube

convgru_ensemble/datamodule.py (condensed)

```
def __getitem__(self, idx):  
    t0, x0, y0 = self.coords.iloc[idx]          # one CSV row  
    t = self.rng.integers(t0, t0 + 24 - steps + 1)  
    cube = self.RR[t:t+steps, x0:x0+256, y0:y0+256]  
    return rainrate_to_normalized(cube)          # mm/h → [-1, 1]
```

- `__len__`: number of sampled cubes
- `__getitem__`: read, normalise, mask
- Random time window inside each 24-step cube
- Reads `steps` full chunks, then crops 256×256
- Train only: random rotations and flips

See it in practice → [Notebook 04 • 1. Data](#)

LightningDataModule: splits and loaders

convgru_ensemble/datamodule.py (condensed)

```
def setup(self, stage=None):
    n = len(pd.read_csv(self.csv_path))      # sorted by t
    train_end, val_end = int(n * 0.90), int(n * 0.95)
    self.train_dataset = SampledRadarDataset(
        ..., indices=range(0, train_end), augment=True)
```

Split	Share
train	90 % (oldest)
validation	5 %
test	5 % (newest)

Why chronological?

Neighbouring cubes overlap in time: a random split would leak the same storm into test.

See it in practice → [Notebook 04 • 1. Data](#)

LightningModule: where the training logic lives

Hook	In IRENE
<code>__init__</code>	build EncoderDecoder + loss
<code>forward</code>	past → forecast
<code>training_step</code>	shared_step, 10 members
<code>validation_step</code>	shared_step, 10 members
<code>test_step</code>	shared_step, 10 members
<code>configure_optimizers</code>	Adam + ReduceLROnPlateau

RadarLightningModel.shared_step (excerpt)

```
def shared_step(self, batch, split="train"):
    data = batch["data"] # (B, T, 1, H, W)
    past, future = data[:, :-steps], data[:, -steps:]
    preds = self(past, steps).clamp(-1, 1) # (B, T, M, H, W)
    return self.criterion(preds, future, batch["mask"])
```

See it in practice → [Notebook 04 • 2. Model](#)

Optimization strategies

Choice	Setting	Why
Optimizer	Adam, lr 1e-4 (fused)	robust default for recurrent conv nets
LR schedule	ReduceLROnPlateau, factor 0.5, patience 10	halve lr when val_loss stalls
Early stopping	val_loss, patience 100	long patience: noisy ensemble loss
Checkpoints	best val_loss and best train_loss_epoch	keep both, pick later
Batch	32 cubes $\times$ 18 steps (6 past + 12 future)	fits GPU memory at 256×256
Augmentation	random 90° rotations and flips	no preferred storm direction in the data
Ensemble	10 members in training and validation	afCRPS needs a spread to score
Split	chronological 90 / 5 / 5 %	no leakage between nearby storms

See it in practice → [Notebook 04 • 4. Optimization strategies](#)

Configuring and launching a run

full training run (GPU, hours to days)

```
uv run python -m convgru_ensemble.train \
  --config config:experiment \
  --config set:model.num_blocks=5 \
  --config set:model.forecast_steps=12 \
  --config set:model.loss_class=crps \
  --config set:datamodule.batch_size=16 \
  --config set:trainer.max_epochs=100
```

```
uv run tensorboard --logdir logs/
```

today, in 04_training_pipeline_demo.ipynb

```
trainer = Trainer(max_epochs=1, limit_train_batches=3,
                  limit_val_batches=1, accelerator="cpu")
```

Fiddle ≈ Hydra

Hydra: YAML files + key=value overrides.

Fiddle: Python config objects + set: overrides.

Same goal: reproducible, versioned experiments.

What the demo proves

Data → model → afCRPS → gradients → optimizer step, with shrunken sizes, and the logged train / val loss curves. Not a trained model.

See it in practice → [Notebook 04 · 3. Trainer](#)

In conclusion

- **Nowcasting** needs ensembles: uncertainty grows within minutes
- **IRENE = ConvGRU encoder-decoder + noisy decoder, trained with almost-fair CRPS** = ConvGRU encoder-decoder + noisy decoder, trained with CRPS
- **Data selection matters as much as architecture:** NaN filtering + importance sampling
- **uv · PyTorch · Lightning · Fiddle** make the pipeline reproducible and readable
- **IRENE beats STEPS and DGMR on CRPS out to 2 h** — and MAE alone would have told you otherwise

References and credits

- Camilletti, A., Franch, G. et al. — IRENE: A Convolutional GRU Ensemble Model for Radar Precipitation Nowcasting over Italy (in preparation) — architecture, comparison and score figures
- Franch, G. et al. (2026). IT-DPC-SRI: A Cloud-Optimized Archive of Italian Radar Precipitation (2010–2025) — doi.org/10.5281/zenodo.18637608
- DGMR baseline: Google DeepMind, [deepmind-research/nowcasting](https://deepmind-research.github.io/nowcasting/) (2021), used as released
- IRENE model card and ConvGRU-Ensemble code — huggingface.co/it4lia/irene · github.com/DSIP-FBK/ConvGRU-Ensemble
- Franch, G. et al. (2020) — encoder–forecaster radar nowcasting (cited in the original hands-on slides)
- Bowler, N. E., Pierce, C. E. & Seed, A. W. (2006). STEPS: a probabilistic precipitation forecasting scheme. *Q. J. R. Meteorol. Soc.*, 132, 2127–2155
- Pulkkinen, S. et al. (2019). Pysteps: an open-source Python library for probabilistic precipitation nowcasting. *Geosci. Model Dev.*, 12, 4185–4219
- Lang, S. et al. (2024). AIFS-CRPS — almost fair CRPS (afCRPS)
- Radar data: DPC rete RADAR, processed by FBK — CC BY-SA 4.0 · Nowcasting comparison image: deepmind.google
- Slides adapted from R. Wanjari, "Modern Deep Learning Tutorial" (27/10/2025) and A. Camilletti, "Hands-on nowcasting" (29/10/2025)

APPENDIX

Appendix

Material kept for reference: model internals, the STEPS baseline and the notebook-based benchmarking session.

Building block: the ConvGRU cell

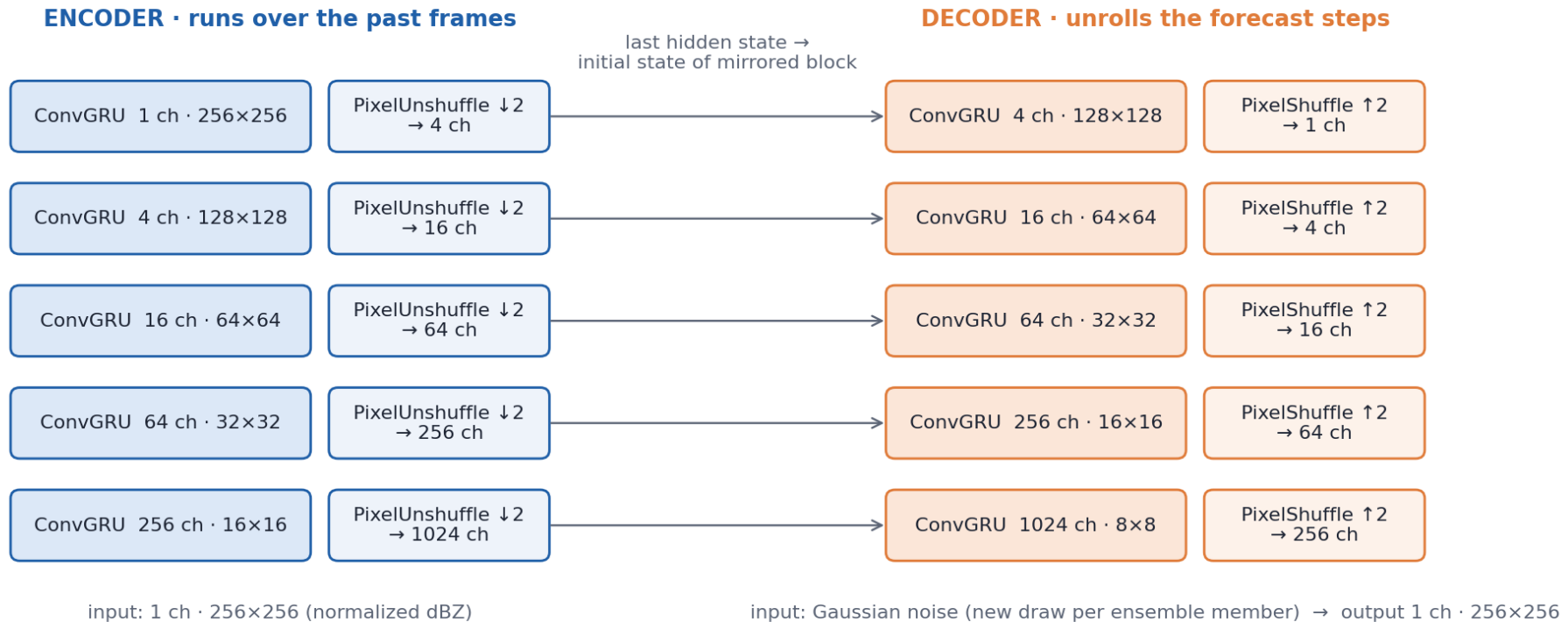
$$\begin{aligned} z, r &= \sigma(\text{Conv}[x, h]) \\ \tilde{h} &= \tanh(\text{Conv}[x, r \odot h]) \\ h' &= (1 - z) \odot h + z \odot \tilde{h} \end{aligned}$$

- **z** update gate • **r** reset gate
- Convolutions keep the spatial structure of the state
- 3×3 kernels: local interactions, stacked over scales

```
convgru_ensemble/model.py
class ConvGRUCell(nn.Module):
    def forward(self, x, h):
        update, reset = self.gates(x, h) # z, r
        h_new = torch.tanh(self.out_gate(x, h * reset))
        return h * (1 - update) + h_new * update
```

See it in practice → [Notebook 02 • 8. Where each piece shows up in IRENE](#)

Architecture: 5 encoder blocks, 5 decoder blocks



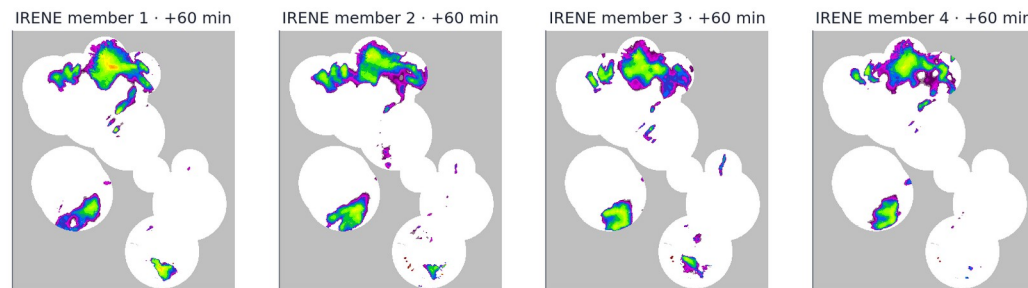
- **PixelUnshuffle ↓2**: H/2, W/2, channels ×4 — lossless, unlike pooling. **Decoder runs coarse → fine**: 8×8 state first, full resolution last.

Ensembles from a noisy decoder

EncoderDecoder.forward (excerpt)

```
encoded = self.encoder(x)                # runs once
for _ in range(ensemble_size):
    x_dec = torch.randn(x_dec_shape)     # new noise
    preds.append(self.decoder(x_dec, last_hidden))
return torch.cat(preds, dim=2)           # (B, T, M, H, W)
```

- Encoder runs once
- Decoder runs M times, each with new Gaussian input
- Spread between members = forecast uncertainty
- **Not Bayesian:** spread is learned through the CRPS loss



See it in practice → [Notebook 01 · 6. Why an ensemble?](#)

Baselines: persistence and STEPS

Persistence

Future = last observed frame. The floor any nowcasting method has to beat.

pysteps – same past frames as IRENE

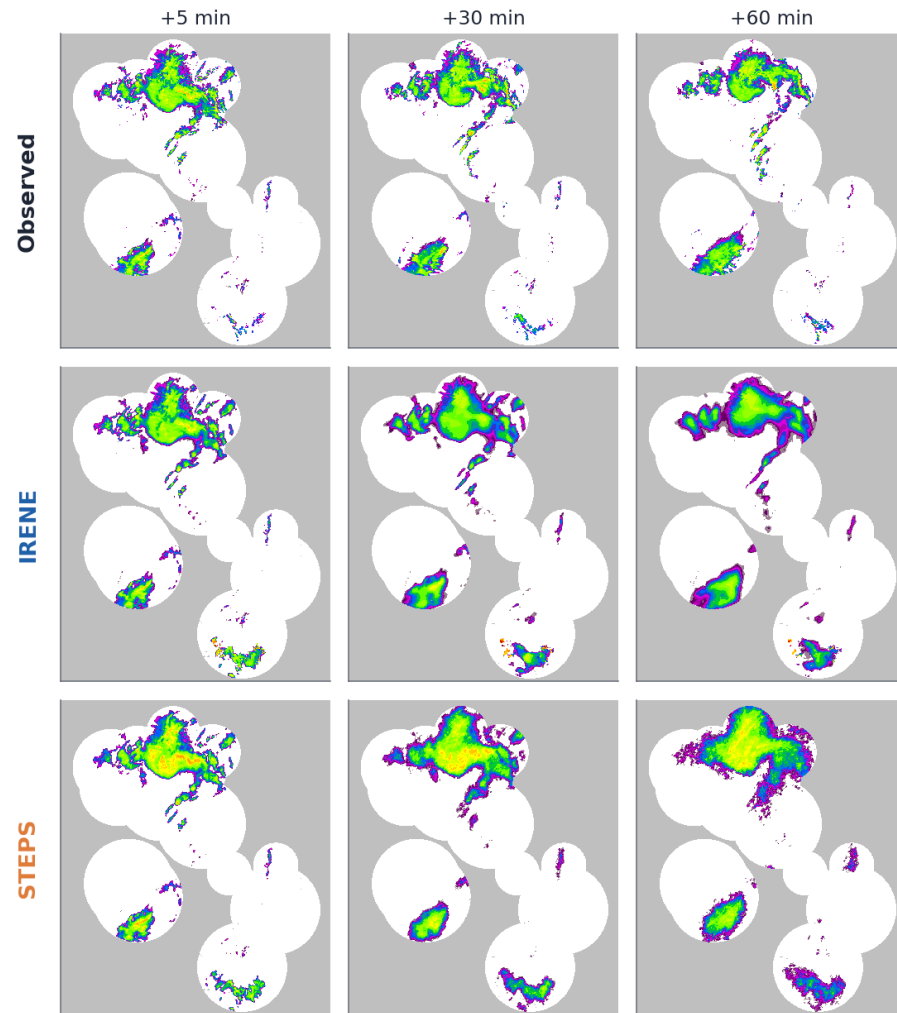
```
R_db, meta = transformation.dB_transform(past, metadata)
V = motion.get_method("lucaskanade")(R_db)
fc_db = nowcasts.get_method("steps")(
    R_db[-3:], V, timesteps=12, n_ens_members=6, ...)
fc, _ = transformation.dB_transform(fc_db, meta, inverse=True)
```

STEPS

- 1 • motion by optical flow
- 2 • cascade decomposition by scale
- 3 • AR(2) evolution per scale
- 4 • stochastic noise → ensemble

See it in practice → [benchmarking_steps.ipynb](#) • 2. The STEPS baseline

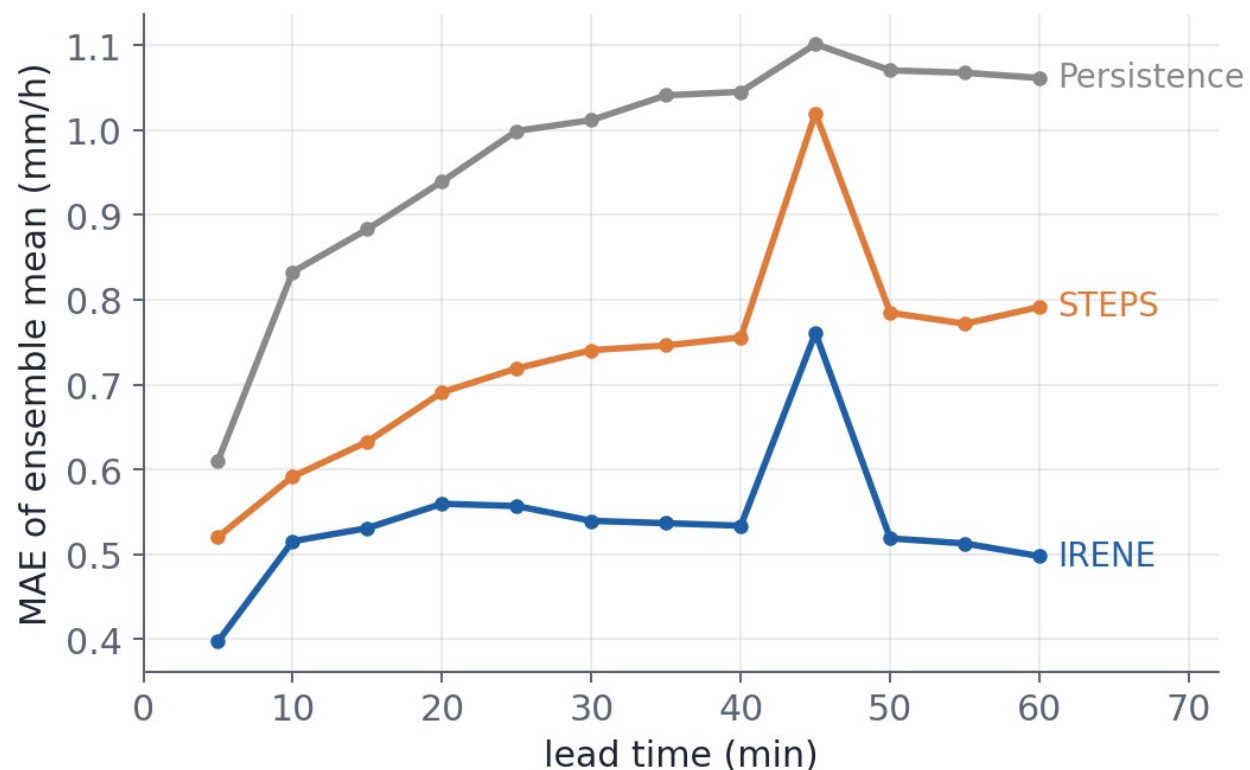
Side by side: 28 August 2025, 22:30 → 23:25 UTC



- **Same input:** 7 past frames up to 22:25 UTC
- **Same size:** 6 ensemble members each
- **STEPS** moves the observed storm along its estimated motion
- **IRENE** lets intensity and extent evolve over the hour
- **Ensemble means** are smoother than any member — judge members too

See it in practice → [benchmarking_steps.ipynb](#) • [3. Comparison movie](#)

Error vs lead time



One event, ensemble-mean MAE only. Proper verification: many events, CRPS, FSS (pysteps.verification).

MAE (mm/h)	+5 min	+30 min	+60 min
IRENE	0.40	0.54	0.50
STEPS	0.52	0.74	0.79
Persistence	0.61	1.01	1.06

Spike at +45 min?

All three models jump together: the observed 23:10 frame is ~45 % wetter than its neighbours — likely an artefact in the ground truth, not the models.

See it in practice → [benchmarking_steps.ipynb](#) • 4. MAE vs lead time

Setup

```
cd seminars/module2-irene-nowcasting
uv sync

bash scripts/download_checkpoint.sh    # ~735 MiB

uv run jupyter lab

# then: notebooks/00_setup_check.ipynb → Run All
```

```
alternative: download on first use from Hugging Face
model = RadarLightningModel.from_pretrained("it4lia/irene")
```

Checkpoint

~735 MiB, not in git
Start the download during the break

Credentials

.env at the repository root, shared with
Module 1 — needed for the live-archive
notebooks only.

See it in practice → [Notebook 00 • Setup check](#)

Three ways to run IRENE

1 · Python

```
radar = xr.open_dataarray("examples/sample_data.nc")
past = radar[:7]

model = RadarLightningModel.from_checkpoint(
    "checkpoints/model.ckpt")
pred = model.predict(past, forecast_steps=12,
                    ensemble_size=10)
```

2 · Command line

```
uv run convgru-ensemble predict \
    --input examples/sample_data.nc \
    --hub-repo it4lia/irene \
    --forecast-steps 12 \
    --ensemble-size 10 \
    --output predictions.nc
```

3 · HTTP API (FastAPI)

```
uv sync --extra serve; uv run convgru-ensemble serve --hub-repo it4lia/irene --port 8000
curl -X POST "localhost:8000/predict?forecast_steps=12&ensemble_size=5" \
    -F "file=@examples/sample_data.nc" -o predictions.nc
```

See it in practice → [Notebook 01 · 3. From the archive to predict\(\)](#)

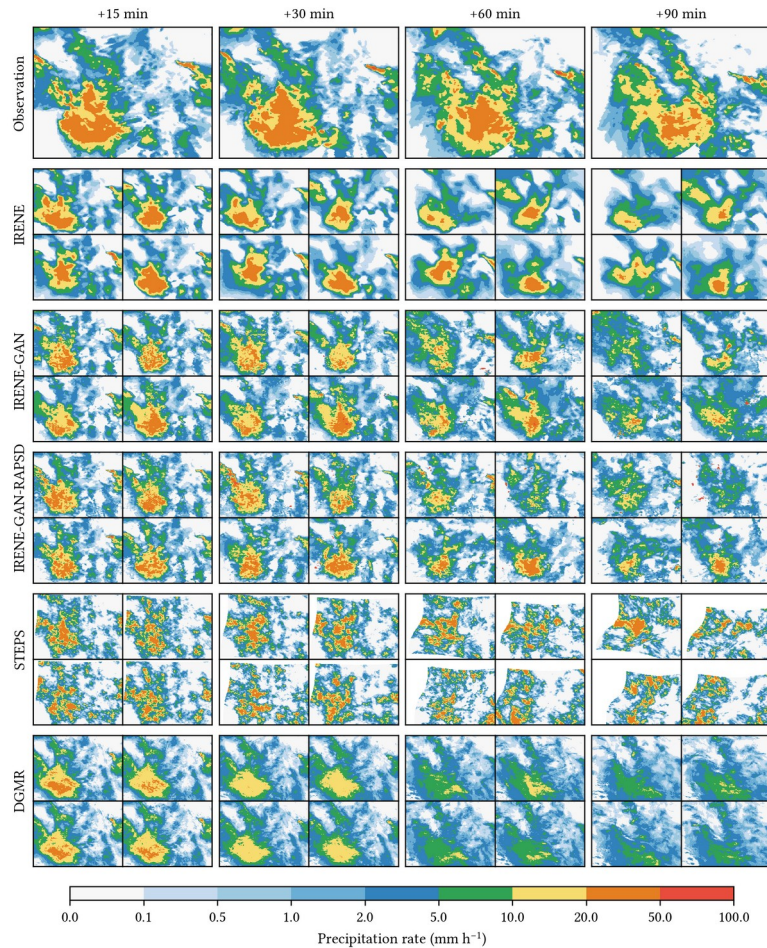
Notebooks

Notebook	Agenda slot	You will
<code>00_setup_check.ipynb</code>	Before you start	Check the kernel, the packages, the checkpoint and ArcoDataHub access
<code>01_irene_live_inference.ipynb</code>	Hands-on: how to use IRENE	Load IRENE, stream live radar, predict, and use the ensemble
<code>02_pytorch_dl_intro.ipynb</code>	PyTorch foundations	Tensors, autograd, nn.Module, DataLoaders, Lightning
<code>03_importance_sampling_demo.ipynb</code>	Training pipeline	NaN cleaning and importance sampling on the live archive
<code>04_training_pipeline_demo.ipynb</code>	Training pipeline	DataModule + LightningModule + Trainer, a few real gradient steps
<code>benchmarking_steps.ipynb</code>	not run in this edition	IRENE vs STEPS vs persistence: movie and MAE curve

Exercises

- **1 • Horizon.** Forecast 24 steps (2 h) instead of 12. Where does IRENE's error overtake STEPS', if ever?
- **2 • Ensemble size.** Compare 2, 5 and 10 members: how do the ensemble mean and the spread change?
- **3 • Sampling.** Rerun the importance sampler with a stricter `n_nan` and `m = 3`. What fraction of cubes survives?
- **4 • Loss.** In the training demo switch `loss_class` to `mae`, then `afcrps`. What happens to `train_ensemble_std`?
- **5 • STEPS knobs.** Try `n_cascade_levels = 3` and fewer past frames. Who suffers more from less history?
- **6 • Better scores.** Add FSS at 1 mm/h with `pysteps verification`. Does the ranking match MAE?

All five models, four members each



IRENE paper (Camilletti, Franch et al., in preparation)

- **IRENE** — probabilistic, CRPS-type loss: the released checkpoint
- **IRENE-GAN** — adversarial training: sharper, more textured members
- **IRENE-GAN-RAPSD** — adversarial plus a spectral constraint
- **STEPS** — optical-flow extrapolation with stochastic noise (pysteps)
- **DGMR** — DeepMind's pretrained generative nowcaster, applied as-is
- All three IRENE variants beat STEPS and DGMR on CRPS at every lead time out to 120 min